PostgreSQL 11 → 16 Migration Runbook

Standalone PG11 (WebFOCUS repository) → the db01 / db02 PG16 streaming-replication cluster
Globals + database (excluding **public.botldata** data) + size-filtered **botldata** reload

Field	Value
Source	`PostgreSQL 11 standalone, single database (WebFOCUS repo)`
Target primary	`db01 (PostgreSQL 16)`
Target standby	`db02 (streams via slot ssncpri — do not restore here)`
Clients	`Dump: source v11 (/usr/pgsql-11/bin) · Restore: db01 v16 (/usr/pgsql-16/bin)`
Table handled separately	`public.botldata (column: report)`
Row filter	`octet_length(report) < 20*1024*1024 (rows under 20 MB)`
FK	`public.botldata(reportid) → public.botlib(reportid)`
Staging directory	`/apps/backups/pg11to16`
To fill in	`<SRC_HOST> = PG11 host/IP · <SRC_DB> = repo DB name`

0 Scenario & design decisions

- **Dump with the source's own v11 client, restore with v16.** A newer pg_restore reads an older archive (forward-compatible); the reverse is *unsupported*, so the load side stays on `/usr/pgsql-16/bin` on db01. Because a v11 dumper can still emit constructs v16 removed (OID-era syntax is the classic 11→12 casualty), the artifacts are scanned before loading — see §5.0.
- **--exclude-table-data, not --exclude-table.** The botldata definition, PK, indexes and its FK stay in the main dump in correct dependency order; only its rows are omitted, then reloaded filtered. This sidesteps the dependency-ordering traps of fully excluding the table.
- **Row filtering needs \copy, not pg_dump.** pg_dump has no row-level WHERE in any version, so the <20 MB slice is exported with `\copy (SELECT … WHERE …)`.
- **Restore on the primary only.** db01 receives everything; db02 replays it through the ssncpri slot automatically.

> **Why a v11 dump restores into v16 (with one scan)**
> pg_restore is forward-compatible: v16 reads a v11 archive cleanly, but v11 must never restore into v16. The only wrinkle is that the v11 dumper can emit syntax v16 dropped (OID-era constructs), so §5.0 scans the artifacts before loading; the source-side WITH OIDS check in §1.3 catches the same thing at origin.

1 Pre-flight checks

Run §1.1–1.4 on the source (they use its v11 client). Use a `~/.pgpass` (chmod 600) or `PGPASSWORD` so nothing prompts. Adjust `/usr/pgsql-11/bin` to your source's v11 client path if it differs.

1.1 Confirm the clients and both servers answer

```
# on the SOURCE -- its own v11 client:
/usr/pgsql-11/bin/pg_dump --version
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "SELECT version();"

# on db01 -- the v16 client that will do the restore:
/usr/pgsql-16/bin/pg_restore --version
```

1.2 Drop amcheck on the source so it is not carried into the dump

```
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "DROP EXTENSION IF EXISTS amcheck;"
```

1.3 Eyeball remaining extensions and legacy WITH OIDS tables

```
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c "\dx"
# WITH OIDS was removed in PG12 -- this should return zero rows:
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "SELECT relname FROM pg_class WHERE relhasoids;"
```

1.4 FK sanity check — confirm nothing references botldata

```
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"SELECT conrelid::regclass FROM pg_constraint \
 WHERE contype='f' AND confrelid='public.botldata'::regclass;"
```

> **Expected: zero rows**
> botldata is the **child** (it points to botlib), so nothing downstream should depend on it. Zero rows means the filtered reload is safe and no sectioned restore is needed. If rows come back, stop — see §2.

> **Capacity before you start**
> The <20 MB botldata slice is written as an uncompressed CSV — make sure `/apps/backups` has room. The restore also generates a lot of WAL; given this pair once drove slot ssncpri to `lost`, watch archive/disk headroom and db02 replay lag throughout.

2 Referential-integrity analysis (why the filter is safe)

The only constraint on the table is `public.botldata(reportid) → public.botlib(reportid)`, so botldata is the child and botlib the parent. Dropping the >20 MB botldata rows removes child rows only:

- botlib (parent) is migrated in full by the main dump, so every surviving botldata row still points to a valid parent — no orphans.
- In the single pg_restore pass the FK is (re)created during post-data while botldata is still empty (trivially valid); the filtered rows load afterward and are validated one-by-one against the fully-loaded botlib.
- Nothing references botldata, so discarding the large rows breaks nothing downstream.

If §1.4 returned any table

Then some table *is* a child of botldata and filtering its rows would orphan those children. Do not run the simple flow: switch to a sectioned restore (`--section=pre-data` → `--section=data` → load botldata → `--section=post-data`) and decide how to handle the orphaned children first.

3 Source-side backups (v11 client)

3.0 Staging directory

```
mkdir -p /apps/backups/pg11to16
```

3.1 Globals first (roles, tablespaces)

```
/usr/pgsql-11/bin/pg_dumpall -h <SRC_HOST> -p 5432 -U postgres \
  --globals-only -f /apps/backups/pg11to16/01_globals.sql
```

3.2 Database dump — botldata structure kept, its data excluded

```
/usr/pgsql-11/bin/pg_dump -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  --create \
  --exclude-table-data=public.botldata \
  -Fc -v \
  -f /apps/backups/pg11to16/02_maindb_no_botldata_data.dump
```

For a large repo you can swap `-Fc` for `-Fd -j 4` (directory format, parallel dump).

3.3 The botldata rows under 20 MB on the report column

```
/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"\copy (SELECT * FROM public.botldata \
 WHERE octet_length(report) < 20*1024*1024) \
 TO '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"
```

> **octet_length assumes report is bytea / text**
> It counts stored bytes — exactly the 20 MB test you want. If report is actually a large object (lo / oid), \copy would export only the OID, not the content; that needs a different approach.

4 Transfer to the primary

Skip this if you are running the dumps *from* db01 — the files are already local.

```
scp -r /apps/backups/pg11to16 postgres@db01:/apps/backups/
```

5 Restore on the primary (db01, v16)

5.0 Pre-load compatibility scan (before restoring)

Because the archive was produced by v11, scan it with the v16 pg_restore for OID / removed-GUC constructs first. All three commands should print *nothing*.

```
# list the archive contents (objects that will be restored):
/usr/pgsql-16/bin/pg_restore -l \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump | less

# scan the DB archive as SQL for removed constructs (expect no matches):
/usr/pgsql-16/bin/pg_restore -f - \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump \
  | grep -niE \
    -e 'with oids' -e 'default_with_oids' -e 'oids *=' \
    -e 'replacement_sort_tuples' -e 'operator_precedence_warning'

# scan the globals the same way (expect no matches):
grep -niE \
  -e 'with oids' -e 'default_with_oids' \
  -e 'replacement_sort_tuples' -e 'operator_precedence_warning' \
  /apps/backups/pg11to16/01_globals.sql
```

If a scan does print a match
Convert that dump to plain SQL (`pg_restore -f fixed.sql …`), delete the offending line(s), and load with `psql -f fixed.sql` instead of pg_restore. For a clean WITH-OIDS-free PG11 repo (per §1.3) you should not hit this.

Never restore on db02
All restore steps target db01. The standby db02 replays the changes through the ssncpri slot on its own — touching it directly will break replication.

5.1 Globals

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres \
  -f /apps/backups/pg11to16/01_globals.sql
```

“role / tablespace already exists” errors here are harmless.

5.2 Database (creates the DB, botldata present but empty)

```
/usr/pgsql-16/bin/pg_restore -h db01 -p 5432 -U postgres -d postgres \
  --create -Fc -v -j 4 \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump
```

5.3 Load the filtered botldata rows

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"\copy public.botldata \
 FROM '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"
```

6 Post-restore validation

6.1 Refresh planner statistics (none exist until analyzed)

```
/usr/pgsql-16/bin/vacuumdb -h db01 -p 5432 -U postgres -d <SRC_DB> \
  --analyze-in-stages
```

6.2 Verify row counts (botldata should be smaller than source)

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"SELECT count(*) FROM public.botldata;"
```

6.3 Replication health

```
# on the primary: is db02 streaming and caught up?
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres -c \
"SELECT client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"

# on the standby: should return t, counts should match the primary
/usr/pgsql-16/bin/psql -h db02 -p 5432 -U postgres -d postgres -c \
"SELECT pg_is_in_recovery();"
```

7 Rollback / re-run

- The target is a fresh restore, so re-running is clean: on db01 drop the half-restored DB (`DROP DATABASE <SRC_DB>;` from the postgres DB, after disconnecting sessions) and repeat from §5. The source is untouched throughout.
- If only the botldata load failed, you do not need to redo §5.2 — `TRUNCATE public.botldata;` on db01 and re-run §5.3.
- Keep the three artifacts in /apps/backups/pg11to16 until db02 shows the matching count and zero lag.

Appendix A Full sequence (quick copy)

Every long command is wrapped with backslash continuations, so each block still pastes and runs as a single command. Dump side is v11; restore side is v16.

Source (v11 client)

```
mkdir -p /apps/backups/pg11to16

/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "DROP EXTENSION IF EXISTS amcheck;"

/usr/pgsql-11/bin/pg_dumpall -h <SRC_HOST> -p 5432 -U postgres \
  --globals-only -f /apps/backups/pg11to16/01_globals.sql

/usr/pgsql-11/bin/pg_dump -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  --create --exclude-table-data=public.botldata -Fc -v \
  -f /apps/backups/pg11to16/02_maindb_no_botldata_data.dump

/usr/pgsql-11/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"\copy (SELECT * FROM public.botldata \
 WHERE octet_length(report) < 20*1024*1024) \
 TO '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"

scp -r /apps/backups/pg11to16 postgres@db01:/apps/backups/
```

Primary — db01 (v16): scan, then restore

```
# pre-load scan -- all should print nothing
/usr/pgsql-16/bin/pg_restore -f - \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump \
  | grep -niE -e 'with oids' -e 'default_with_oids' -e 'oids *=' \
    -e 'replacement_sort_tuples' -e 'operator_precedence_warning'

/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres \
  -f /apps/backups/pg11to16/01_globals.sql

/usr/pgsql-16/bin/pg_restore -h db01 -p 5432 -U postgres -d postgres \
  --create -Fc -v -j 4 \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump

/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"\copy public.botldata \
 FROM '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"

/usr/pgsql-16/bin/vacuumdb -h db01 -p 5432 -U postgres -d <SRC_DB> \
  --analyze-in-stages
```